MODULE-4

Complexity Theory: Class P and NP, Polynomial time reductions, Class NP Hard and NP Complete, Example Problems- Vertex Cover problem, Clique Problem. Network Flows: Flow Networks and Network Flow, Max- Flow Min Cut Theorem, Ford Fulkerson method.

Introduction to Complexity Theory

Complexity theory is a branch of theoretical computer science that studies the difficulty of computational problems and the amount of resources required to solve them. When we design an algorithm, it is not sufficient to know that the algorithm produces the correct answer. We must also consider how efficiently it can solve the problem, especially when the size of the input becomes very large. Complexity theory mainly studies two important resources: time and space. Time complexity describes how the running time of an algorithm increases as the input size increases, while space complexity describes how much memory is required.

Features of Complexity Theory

1. Studies Problem Difficulty

- Complexity theory studies how difficult a computational problem is.
- It determines whether a problem can be solved efficiently or requires a large amount of computation.

2. Measures Computational Resources

- It mainly measures the resources required by an algorithm.
- The two major resources are:
    - Time
    - Space (memory)

3. Depends on Input Size

- Complexity is studied based on the size of the input.
- Usually, input size is represented by (n).
- We study how the required time or memory changes when (n) increases.

4. Uses Asymptotic Analysis

- Complexity is commonly expressed using:
  - Big-O (O)
  - Big-Omega
  - Big-Theta
- These notations describe the growth rate of an algorithm.

5. Classifies Computational Problems

Complexity theory groups problems into different classes, such as:

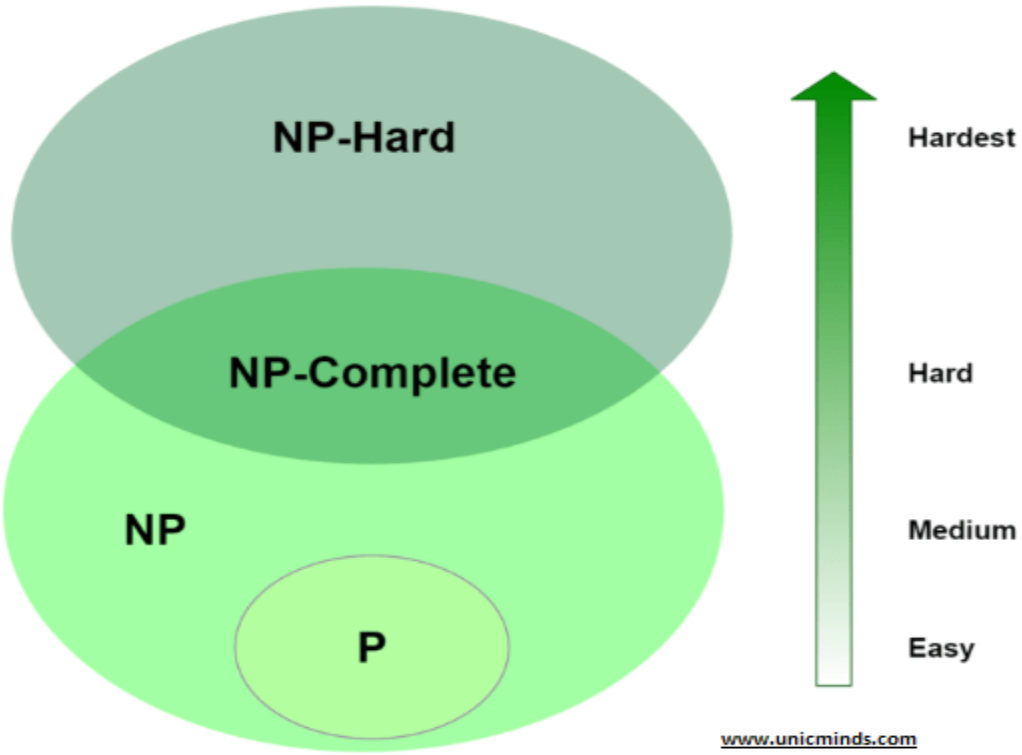
- P
- NP
- NP-Hard
- NP-Complete

6. Uses Polynomial-Time Reduction

- Polynomial-time reduction is used to compare the difficulty of two problems.
- It is especially important for proving that problems are NP-Hard or NP-Complete.

CLASS P AND CLASS NP

Class P and Class NP are two important classes in complexity theory used to classify computational problems based on how efficiently they can be solved or verified. **Class P (Polynomial Time)** contains problems that can be solved by an algorithm in polynomial time, such as (O(n)), (O(n^2)), or (O(n^3)), and these problems are generally considered efficiently solvable. **Class NP (Nondeterministic Polynomial Time)** contains problems for which a given solution can be verified in polynomial time, even if finding the solution itself may be difficult. For example, consider a **Sudoku problem**. Finding the correct solution to a difficult Sudoku puzzle may take considerable time, but if someone gives us a completed Sudoku, we can quickly check whether all rows, columns, and boxes contain the required numbers without repetition. The relationship between the two classes is (P is a subset of NP), because a problem that can be solved quickly can also be verified quickly. One of the most important unanswered questions in computer science is whether (P=NP).



Class P

What is Class P?

P stands for Polynomial Time.
Class P is a group of decision problems that can be solved by an algorithm in polynomial time.
Class P contains problems that can be solved efficiently, even when the input size becomes large.
The word polynomial refers to the running time of the algorithm.

What is Polynomial Time?

If the running time of an algorithm is represented by:
O(n^k)
where:

- (n) = size of the input
- (k) = a constant

then the algorithm is said to run in polynomial time.

Examples

The following are polynomial-time complexities:
O(n)
O(n^2)

O(n^3)
O(n^4)
For example, if an algorithm takes:
O(n^2)
and the input size is (n=100):
100^2=10,000
operations are required approximately.

Why is Class P Important?

Class P helps us identify problems that can be solved **efficiently**.
Suppose we have two algorithms:

Algorithm A

O(n^2)

Algorithm B

O(2^n)

For a small input, both may work.
But when the input becomes very large, (2^n) increases extremely quickly.
Therefore, polynomial-time algorithms are generally considered **practical and efficient**, while exponential-time algorithms can become impractical for large inputs.

Class P Contains Decision Problems

In complexity theory, P is usually defined using **decision problems**.
A decision problem has only two possible answers: yes or no

Example

Consider a graph.
Question:
"Is there a path between vertex A and vertex B?"
The answer is either:

- YES
- NO

If this problem can be solved in polynomial time, it belongs to P.

Simple Example – Searching

Suppose we have a list:
10,\ 25,\ 15,\ 40,\ 30
and we want to know:
"Is 40 present in the list?"
We can check the elements one by one.
The time complexity is:
O(n)
Since (O(n)) is polynomial time, this type of problem belongs to P.

Another Example – Shortest Path

Suppose several cities are connected by roads.
We want to find whether there is a path between two cities or find the shortest path.
Many graph problems, including important shortest-path problems, have polynomial-time algorithms.
For example, Dijkstra's algorithm can solve shortest-path problems under its usual conditions.
Therefore, such problems are examples of efficiently solvable problems.

Examples of Problems in P

Some commonly studied problems with polynomial-time algorithms include:

- Searching
- Sorting
- Binary search
- Graph connectivity
- Shortest path
- Minimum spanning tree
- Maximum flow
- Finding the greatest common divisor

The exact complexity depends on the particular algorithm used.

Main Characteristics of Class P

1. Polynomial-Time Solution

Problems in P can be solved using an algorithm whose running time is polynomial in the input size.

2. Efficiently Solvable

They are generally considered **efficiently solvable**.

3. Input Size Matters

The running time is measured according to the size of the input.

4. Deterministic Algorithms

P is normally defined using **deterministic computation**.
This means that at every step, the algorithm has a definite next step.

5. Solutions Can Also Be Verified Quickly

If a problem can be solved in polynomial time, its solution can also be verified in polynomial time.

P vs Non-Polynomial Time

Consider these two complexities:
O(n^2)
and
O(2^n)

The first one is polynomial.
The second one is exponential.
For example, when (n=20):
n^2=400
whereas:
2^ 20=1,048,576
So the exponential algorithm requires far more operations.
This is why polynomial-time algorithms are generally preferred for large inputs.

Class NP

NP (Nondeterministic Polynomial time) is a class of computational problems in which, if a possible solution is given, we can verify whether that solution is correct in polynomial time. The important idea is that NP does not necessarily mean that finding the solution is easy; rather, it means that checking a proposed solution is efficient. For example, consider the Subset Sum problem. If we are given a set of numbers and a target value, finding a subset whose sum equals the target may be difficult when the number of elements is large. However, if someone gives us a particular subset, we can simply add its elements and check whether the sum is equal to the target. Since this verification can be performed in polynomial time, the problem belongs to NP. Other examples include Hamiltonian Cycle, Clique, Vertex Cover, SAT, Graph Coloring, and the decision version of Travelling Salesman Problem.

The relationship between P and NP is very important. P (Polynomial time) contains problems that can be both solved efficiently and verified efficiently, while NP contains problems whose proposed solutions can be verified efficiently. Therefore, every P problem is also an NP problem, which is written as $P \subseteq NP$. However, it is still unknown whether $P = NP$ or $P \neq NP$. Problems that are both in NP and NP-Hard are called NP-Complete problems, whereas problems at least as difficult as all NP problems are called NP-Hard problems. Thus, a simple way to remember the concept is: P → solution can be found quickly; NP → solution can be checked quickly; NP-Complete → problems that are both NP and NP-Hard.

Polynomial time reductions

A polynomial-time reduction is a method of solving one problem by using another problem. Suppose we have two problems, A and B. If we can convert an input of Problem A into an input of Problem B in polynomial time, and then use a solution of B to obtain the solution of A, we say that A is polynomial-time reducible to B. It is written

as A ≤p B. In simple words, we are saying that Problem A is not harder than Problem B. If we already have an efficient algorithm for solving B, then we can also solve A efficiently by first converting A in to B and then solving B. Polynomial-time reductions are very useful in computational complexity theory because they help us compare the difficulty of different problems and are widely used to identify NP-Hard and NP-Complete problems.

Example: Suppose Problem A is difficult and Problem B is another problem. We take the input of A and transform it into an input of B using a polynomial-time algorithm. Then we call the algorithm for B and use its answer to obtain the answer for A.

Input of A
↓
Polynomial-time transformation
↓
Input of B
↓
Algorithm for B
↓
Answer for A

Therefore, A ≤p B means that A can be solved using B with only polynomial additional work. If B has an efficient algorithm, A will also have an efficient algorithm. Conversely, if A is known to be difficult to solve efficiently and A ≤p B, then B must also be difficult; otherwise, an efficient solution for B would give us an efficient solution for A. This is why reductions are especially important when proving that a new problem is NP-Complete.

Types of Polynomial-Time Reductions

There are three commonly discussed types:

1. Polynomial-time many-one reduction – The input of one problem is transformed directly into an input of another problem. It is the most restrictive and most commonly used type.
2. Truth-table reduction – The first problem can use the answers from several fixed queries to the second problem. It is more flexible than many-one reduction.
3. Turing reduction – The first problem can use the second problem as a subroutine, making multiple queries and using previous answers to decide what query to make next. It is the most general type.

Reduction = Convert one problem into another so that solving the second helps us solve the first.

Types of Polynomial-Time Reductions

There are **three important types of polynomial-time reductions**:

1. **Many-one reduction**
2. **Truth-table reduction**
3. **Turing reduction**

They all have the same basic purpose: **use one problem to solve another problem efficiently**. The main difference is **how the first problem uses the second problem**.

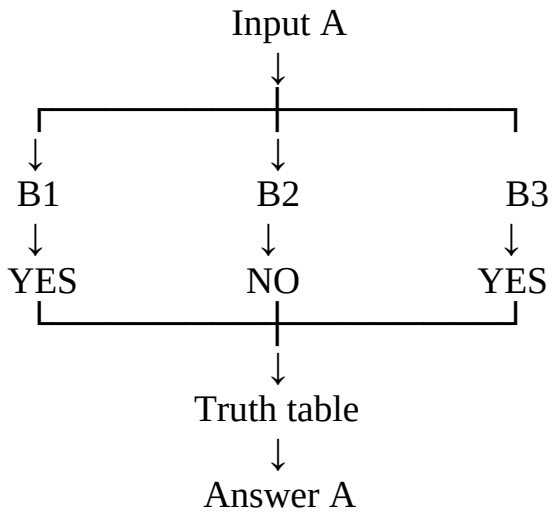
1. Many-One Reduction
A **polynomial-time many-one reduction** converts **one input of Problem A into one input of Problem B** in polynomial time.
It is written as:
$A \leq_p B$
This means:
Problem A can be transformed into Problem B, and the answer remains the same.
Simple process
Input of A
↓
Polynomial-time transformation
↓
Input of B
↓
Solve B
↓
Same answer for A
For example, suppose A asks:
"Is there a solution to problem A?"
We transform its input into problem B.
If:
A = YES → B = YES
and
A = NO → B = NO
then the transformation is valid.
Many-one reduction is also called a **polynomial transformation** or **Karp reduction**.

Easy example

Suppose:
Problem A → Problem B
We convert:
x → y
Then we give y to the algorithm for B.
If B returns **YES**, we say A is **YES**.
If B returns **NO**, we say A is **NO**.
So, **only one call to B is needed**.

2. Truth-Table Reduction

A **truth-table reduction** is more flexible than a many-one reduction.
Here, instead of converting one input of A into only one input of B, we can create **several inputs for B**.
The answers obtained from B are then combined using a fixed rule or **truth table** to obtain the answer for A.



For example, suppose A produces three questions for B:
B1 → YES
B2 → NO
B3 → YES
The answers are combined according to a predetermined rule.
The important point is:
The rule for combining the answers is fixed and does not change for different inputs.
Therefore, a truth-table reduction can make **multiple queries to B**, but the queries are determined without depending on the answers to earlier queries.

3. Turing Reduction

A **polynomial-time Turing reduction** is the **most general** type of reduction.
Here, Problem A can use Problem B as a **subroutine** and can call B **multiple times**.

Most importantly, the next question asked to B **can depend on the answer received from the previous question**.

Problem A
↓
Ask B a question
↓
Get the answer
↓
Decide next question
↓
Ask B again
↓
Get the answer
↓
Final answer

For example:
A
↓
Ask B → YES
↓
Based on YES, ask B another question
↓
B → NO
↓
Based on NO, ask another question
↓
B → YES
↓
Final answer for A

The number of calls to B and the additional computation must be **polynomial**.
Turing reductions are also called **Cook reductions**, named after **Stephen Cook**.

Difference Between the Three

Feature	Many-One	Truth-Table	Turing
Number of calls to B	Exactly **1**	Multiple/fixed number	Multiple
Can later queries depend on earlier answers?	No	No	**Yes**
Flexibility	Least	Medium	**Most**

Other name	Karp reduction	—	Cook reduction
Main idea	One input → one input	One input → several inputs	Use B as a subroutine

NP-Hard

An **NP-Hard problem** is a problem that is **at least as difficult as every problem in NP**. This means that if we can solve an NP-Hard problem quickly, we could also solve all NP problems quickly. To show that a problem is NP-Hard, every NP problem must be convertible to it using a **polynomial-time reduction**. An NP-Hard problem **does not necessarily belong to NP**. It may be an optimization problem rather than a decision problem. For example, in the **Travelling Salesman Problem (TSP)**, a salesman must visit several cities and return to the starting city. Finding the **shortest possible route** is difficult when the number of cities becomes large. Therefore, the optimization version of TSP is an example of an **NP-Hard problem**.

NP-Complete

An **NP-Complete problem** is a problem that has **two important properties**. First, its solution can be **verified in polynomial time**, meaning that if someone gives us an answer, we can check whether it is correct quickly. Second, the problem is also **NP-Hard**, meaning it is at least as difficult as every problem in NP. Therefore, NP-Complete problems are considered the **hardest problems within NP**. For example, **Subset Sum** is an NP-Complete problem. Suppose we have the numbers {3,5,7,10} and the target is 15. We ask whether any subset adds up to 15. Since 3+5+7=15, the answer is YES. We can quickly verify this answer, and Subset Sum is also NP-Hard. Therefore, it is **NP-Complete**.

Differences Between NP-Hard and NP-Complete

No.	NP-Hard	NP-Complete
1.	An NP-Hard problem is **at least as hard as every problem in NP**.	An NP-Complete problem is **both in NP and NP-Hard**.
2.	It **does not have to belong to NP**.	It **must belong to NP**.
3.	Its solution **need not be verifiable in polynomial time**.	Its solution **must be verifiable in polynomial time**.
4.	It can be a **decision or optimization problem**.	It is generally considered a **decision problem**.
5.	It can exist **outside the class NP**.	It is always **within NP**.

6.	It is not necessary to show that the problem belongs to NP.	We must prove that the problem **belongs to NP**.
7.	Example: **TSP optimization**.	Example: **TSP decision problem**.
8.	It may not have a simple YES/NO answer.	It normally has a **YES/NO answer**.
9.	Showing that a known NP problem reduces to it establishes its NP-Hardness.	To prove NP-Completeness, we show **it is in NP + it is NP-Hard**.
10.	NP-Hard is the **broader category**.	NP-Complete is a **subset of NP-Hard**.

Vertex cover problem and clique problem-notes given

Network Flows: Flow Networks and Network Flow

1. Introduction

It is used to solve problems where some amount of a resource has to be moved from one place to another through a network. The resource may be water flowing through pipes, data moving through communication links, goods transported through roads, or electricity moving through a network. Each connection in the network has a limited capacity. The main aim is usually to determine how much flow can be sent from a starting point to a destination without exceeding the capacity of any connection.

A network flow problem is represented using a directed graph. It contains a source, a sink, edges, and capacities. Algorithms such as Ford–Fulkerson and Edmonds–Karp are commonly used to find the maximum possible flow.

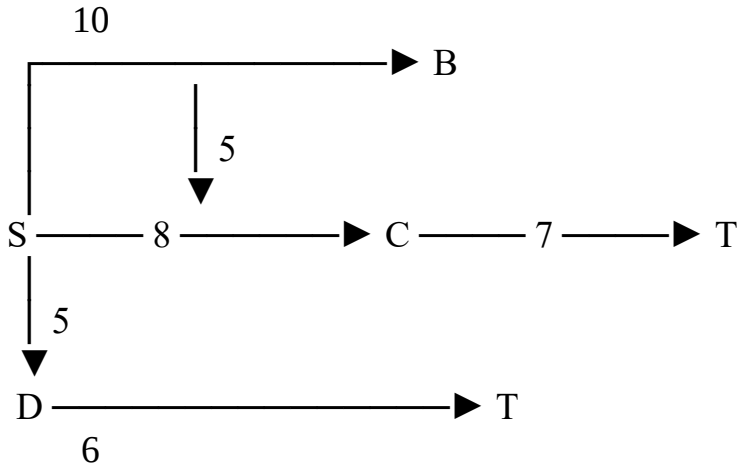
What is a Flow Network?

A flow network is a directed graph in which every edge has a capacity that specifies the maximum amount of flow that can pass through that edge.

A flow network normally contains:

- Vertices (nodes) – represent locations or points.
- Directed edges – represent connections between locations.
- Capacity – maximum flow allowed through an edge.
- Source (s) – starting point of the flow.
- Sink (t) – destination point of the flow.

Simple representation



Here:

- S = Source
- T = Sink
- Numbers on edges = Capacities
- Flow moves in the direction of the arrows.

3. Source

The source is the node from which the flow starts.

It is usually represented by:

ss

For example:

S → A → B → T
↑
Source

Here, S is the source.

Example

If we are transporting goods from a factory to different cities, the factory can be considered the source.

4. Sink

The sink is the final destination where the flow ends.

It is usually represented by:

tt

Example:

S → A → B → T
↑
Sink

If the factory is sending goods to a warehouse, the warehouse can be considered the sink.

5. Capacity

Every edge in a flow network has a capacity.

Capacity means:

The maximum amount of flow that can pass through an edge.

For example:

A ——————▶ B
10

The number 10 means that the maximum flow from A to B is 10 units.

6. Flow

Flow is the actual amount of resource sent through an edge.

Suppose:

A ——————▶ B
10

Capacity = 10.

We may send:

- 2 units
- 5 units
- 8 units
- 10 units

But we cannot send:

- 11 units
- 15 units

because the capacity is only 10.

For example:



Here:

- Capacity = 10
- Actual flow = 7

Flow Conservation

Another important rule is the flow conservation property.

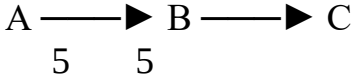
It means:

For every intermediate vertex, the amount of flow entering the vertex must be equal to the amount of flow leaving the vertex.

In simple words:

Whatever comes into a middle node must go out.

Example

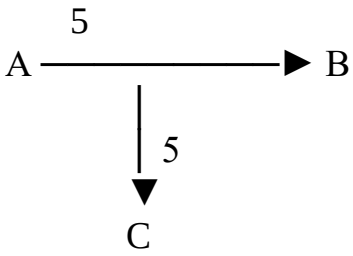


At node B:

- Flow entering = 5
- Flow leaving = 5

Therefore, flow is conserved.

Another example



At B:

Incoming=5Incoming = 5 Outgoing=5Outgoing = 5

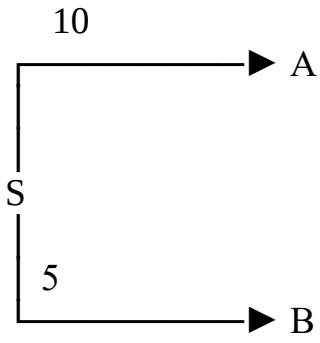
So the flow conservation rule is satisfied.

Flow Value

The value of a flow is the total amount of flow leaving the source.

It is also equal to the total amount of flow entering the sink.

For example:



If 10 units leave S through A and 5 units leave S through B:

Flow Value=10+5=15Flow\ Value = 10+5=15

Therefore, the value of the flow is 15 units.

10. What is Network Flow?

Network flow refers to the actual movement of flow through a flow network while satisfying the rules of the network.

The main objective in many DAA problems is to find the:

Maximum Flow

Maximum flow means:

The largest possible amount of flow that can be sent from the source to the sink without violating any capacity or flow-conservation constraints.

11. Maximum Flow Problem

The maximum flow problem asks:

What is the maximum amount of flow that can be sent from source s to sink t through the given network?

Bottleneck

A bottleneck edge is an edge whose capacity limits the amount of flow that can pass through a path.

Consider:

S ——10—► A ——4—► B ——8—► T

The capacities are:

- S → A = 10
- A → B = 4
- B → T = 8

The smallest capacity is:

Therefore, the maximum flow through this path is 4.

The edge:

A ——4——► B

is the bottleneck.

Easy rule

The smallest capacity on a path determines the maximum flow through that path.

Residual Network

The residual network is an important concept used in maximum-flow algorithms.

It shows how much additional flow can still be sent through each edge.

Suppose:

A ————————► B
capacity = 10
flow = 6

Remaining capacity is:

10−6=410-6=4

Therefore:

A ——4——► B

There is also a reverse edge in the residual network that allows the algorithm to cancel or adjust previously sent flow.

If 6 units were sent from A to B:

A ——4——► B
A ◄——6—— B

The reverse capacity is 6.

This allows the algorithm to change an earlier decision if a better flow arrangement is found.

Augmenting Path

An augmenting path is a path from the source to the sink in the residual network along which additional flow can be sent.

For example:

S ——5——► A ——3——► B ——7——► T

The available capacities are:

- S → A = 5
- A → B = 3
- B → T = 7

The smallest capacity is:

min(5,3,7)=3min(5,3,7)=3

Therefore, we can send 3 more units through this path.

This path is called an augmenting path.

Maximum Flow Algorithms

Two important algorithms for maximum flow are:

1. Ford–Fulkerson Algorithm

It repeatedly:

1. Finds an augmenting path.
2. Finds the minimum remaining capacity on that path.
3. Sends that amount of flow.
4. Updates the residual network.
5. Repeats until no augmenting path is available.

2. Edmonds–Karp Algorithm

Edmonds–Karp is an implementation of Ford–Fulkerson that uses Breadth-First Search (BFS) to find augmenting paths.

It chooses the augmenting path with the fewest number of edges.

Applications of Network Flow

Network flow algorithms are useful in many real-world problems.

Transportation

Finding the maximum amount of goods that can be transported through roads.

Computer Networks

Finding how much data can be transferred through communication links.

Water Distribution

Finding how much water can be transported through a system of pipes.

Traffic Networks

Studying the capacity of roads and transportation networks.

Bipartite Matching

Network flow can be used to match people with jobs, students with projects, or workers with tasks.

Supply Chain

Finding the maximum amount of products that can move from suppliers to customers.

Term	Simple Meaning
Flow network	Directed graph with capacities
Source (s)	Starting point of flow
Sink (t)	Ending point of flow

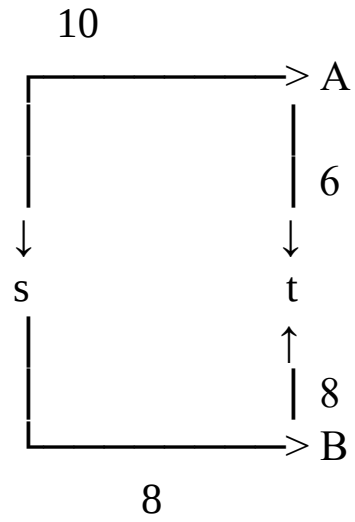
Capacity	Maximum flow allowed on an edge
Flow	Actual amount sent through an edge
Flow conservation	Flow entering a middle node = flow leaving
Maximum flow	Largest possible flow from source to sink
Bottleneck	Smallest capacity limiting a path
Residual network	Network showing remaining possible flow

- A flow network is a directed graph in which each edge has a capacity, with a source and a sink, and a network flow is an assignment of flow to the edges that satisfies capacity and flow-conservation constraints. The maximum-flow problem determines the greatest amount of flow that can be sent from the source to the sink.

Flow Network	Network Flow
It is the structure/network through which flow moves.	It is the actual amount of flow moving through that network.
It consists of vertices, directed edges, source, sink, and capacities.	It assigns a flow value to each edge.
It tells us where the flow can move and how much each edge can carry.	It tells us how much flow is actually being sent.
Example: A pipe system with pipes having capacities.	Example: 5 litres of water actually flowing through a pipe.

Max-flow min-cut theorem

The Max-Flow Min-Cut Theorem states that the maximum amount of flow that can be sent from a source (s) to a sink (t) in a flow network is equal to the capacity of the minimum cut that separates the source from the sink. A maximum flow is the largest possible flow that can travel through the network without exceeding the capacity of any edge, while a cut divides the network into two parts, with the source in one part and the sink in the other. The capacity of a cut is the sum of the capacities of all edges crossing from the source side to the sink side. Among all possible cuts, the one with the smallest capacity is called the minimum cut. Therefore, if the maximum flow through a network is 14 units and the minimum cut has a total capacity of 14 units, then according to the theorem, Maximum Flow = Minimum Cut = 14.



There are two paths from source s to sink t:

- s → A → t → maximum flow = **6 units**
- s → B → t → maximum flow = **8 units**

Therefore, the **maximum flow** is:

6+8=14 units

Now consider the cut just before the sink t. The cut contains the edges:

- A → t with capacity **6**
- B → t with capacity **8**

So, the **minimum cut capacity** is:

6+8=14 units

Therefore:

Maximum Flow=Minimum Cut=14

This example demonstrates the **Max-Flow Min-Cut Theorem**: the maximum flow that can reach the sink is exactly equal to the capacity of the smallest cut separating the source from the sink.

Ford–Fulkerson Method

Introduction

The Ford–Fulkerson method is an algorithm used to find the maximum flow in a flow network.

It determines the maximum amount of flow that can be transported from a starting vertex called the source to a destination vertex called the sink, without exceeding the capacity of any edge.

It is commonly used in DAA to solve maximum-flow problems in graphs.

Example: Imagine a water supply network. Water flows through pipes connecting different places. Each pipe has a maximum capacity. The Ford–Fulkerson method finds the maximum amount of water that can flow from the water tank to the destination.

Important terms

Before understanding the algorithm, learn these basic terms.

Term	Explanation
Flow network	A directed graph in which every edge has a capacity.
Source (s	The vertex from which the flow starts.
Sink (t)	The vertex where the flow ends.
Capacity c(u,v)	The maximum flow allowed through an edge from (u) to (v).
Flow f(u,v)	The actual amount of flow sent through an edge.
Residual capacity	The additional flow that can still be sent through an edge.
Augmenting path	A path from source to sink along which additional flow can be sent.
Maximum flow	The greatest possible total flow from source to sink.

Two important conditions

1. Capacity constraint

The flow on an edge must not exceed its capacity.

2. Flow conservation

For every intermediate vertex, the total incoming flow must equal the total outgoing flow.

The source and sink are exceptions to this intermediate-vertex rule.

Basic idea of the Ford–Fulkerson method

The algorithm works by repeatedly finding an augmenting path from the source to the sink.

For every augmenting path:

1. Find a path from source s to sink t with positive residual capacity on every edge.
2. Find the minimum residual capacity along that path. This is called the bottleneck capacity.
3. Increase the flow along the path by the bottleneck capacity.
4. Update the residual capacities of the edges.

Update Forward and Reverse Residual Capacities

Suppose we have two vertices, A and B, connected by an edge.

Before sending flow

A

→

Capacity = 10

B

The edge A → B has a capacity of 10 units. Initially, no flow has been sent.

Suppose we send 4 units

A

→

4 / 10

Remaining = 6

B

The forward residual capacity is now 10-4=6.

Add a reverse residual edge

A

6 remaining

4 reversible

B

The reverse residual capacity B → A becomes 4.

Why do we update both capacities?

1. Forward residual capacity

This tells us how much more flow we can send in the original direction.

10-4=6

2. Reverse residual capacity

This tells us how much of the previously sent flow can be cancelled if a better path is found.

0+4=4

For example, if a later path uses the reverse edge B → A and sends 2 units, it cancels 2 units of the earlier flow from A → B.

Then the flow from A → B becomes 2 units instead of 4.

Remember this formula

For an original edge A → B:

In simple words: When you send flow forward, the forward edge has less capacity left, while the reverse edge gains the ability to undo that flow.

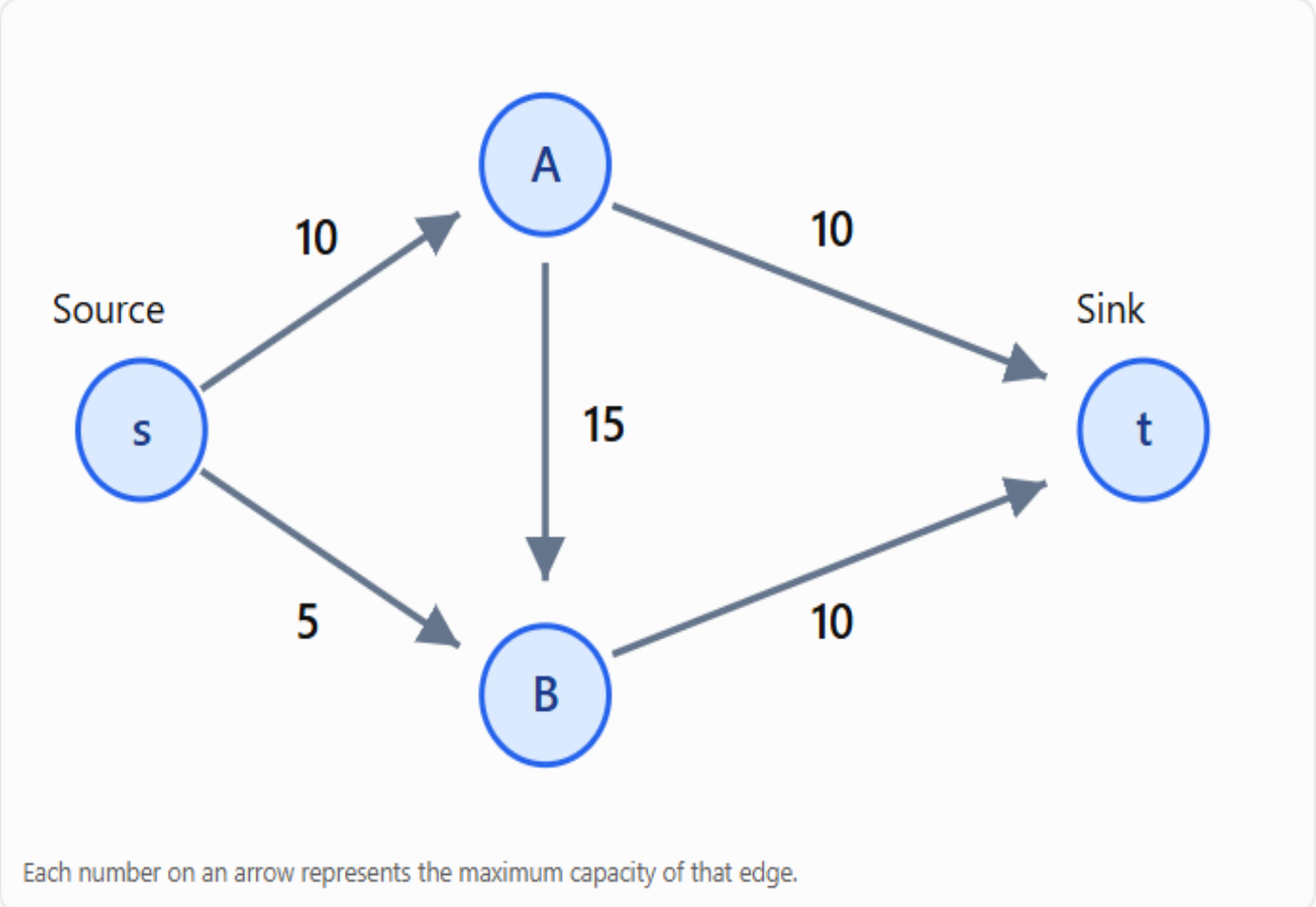
5. Repeat until no augmenting path exists.

The flow added in each iteration is limited by the edge with the smallest available capacity on the selected path.

When no augmenting path remains, the algorithm has found the maximum flow, assuming it terminates under the chosen implementation and capacity conditions.

Step-by-step example

Consider the following flow network.



Edge labels represent capacities. The directed edges are s→A (10), s→B (5), A→B (15), A→t (10), and B→t (10).

Our objective is to find the maximum flow from source (s) to sink (t).

Iteration 1: Find the first augmenting path

Choose the path:

s->A-> ->t

The capacities are:

- s->A=10
- A-> t=10

The bottleneck capacity is the minimum of these values.

min (10,10) =10

Therefore, send 10 units of flow along this path.

After iteration 1

Flow sent

10 units

The edges s→A and A→t is now fully utilized.

Iteration 2: Find another augmenting path

Choose the path: s-> B->t

The capacities are:

- s-> B=5
- B-> t=10

The bottleneck capacity is:

min(5,10)=5

Send 5 additional units of flow.

After iteration 2

Total flow

15 units

The edge s→B is now fully utilized.

Iteration 3: Search for another path

Try to find another path from s to t using edges with positive residual capacity.

- (s-> A) has no remaining capacity.
- (s-> B) has no remaining capacity.

There is no further augmenting path from the source to the sink, so the algorithm stops.

Maximum flow=10+5=15

Notice that the edge A-> B has capacity 15, but it does not need to carry any flow in this solution. The maximum flow is limited by the total capacity leaving the source, which is 10+5=15.

Applications

The Ford–Fulkerson method is useful in:

1. Network routing: Finding the maximum amount of data that can be transferred through a network.
2. Transportation: Maximizing goods transported through roads or pipelines.
3. Bipartite matching: Assigning workers to jobs or students to projects.
4. Resource allocation: Distributing limited resources among different destinations.
5. Communication networks: Identifying the maximum transmission capacity between two points.

- Ford–Fulkerson solves the maximum-flow problem.
- The source is the starting vertex, and the sink is the destination vertex.

- An augmenting path must have positive residual capacity along every edge.
- The bottleneck capacity is the minimum residual capacity along the selected path.
- Residual capacities are updated after every augmentation.
- Reverse edges make it possible to revise previously assigned flow.
- The algorithm terminates when no augmenting path exists, for the usual integral-capacity setting.
- The maximum-flow value equals the capacity of a minimum cut in the network.

The Ford–Fulkerson method finds the maximum flow in a flow network by repeatedly identifying augmenting paths, sending the maximum possible additional flow through each path, and updating the residual graph until no augmenting path remains.